



Programação web

Aula prática

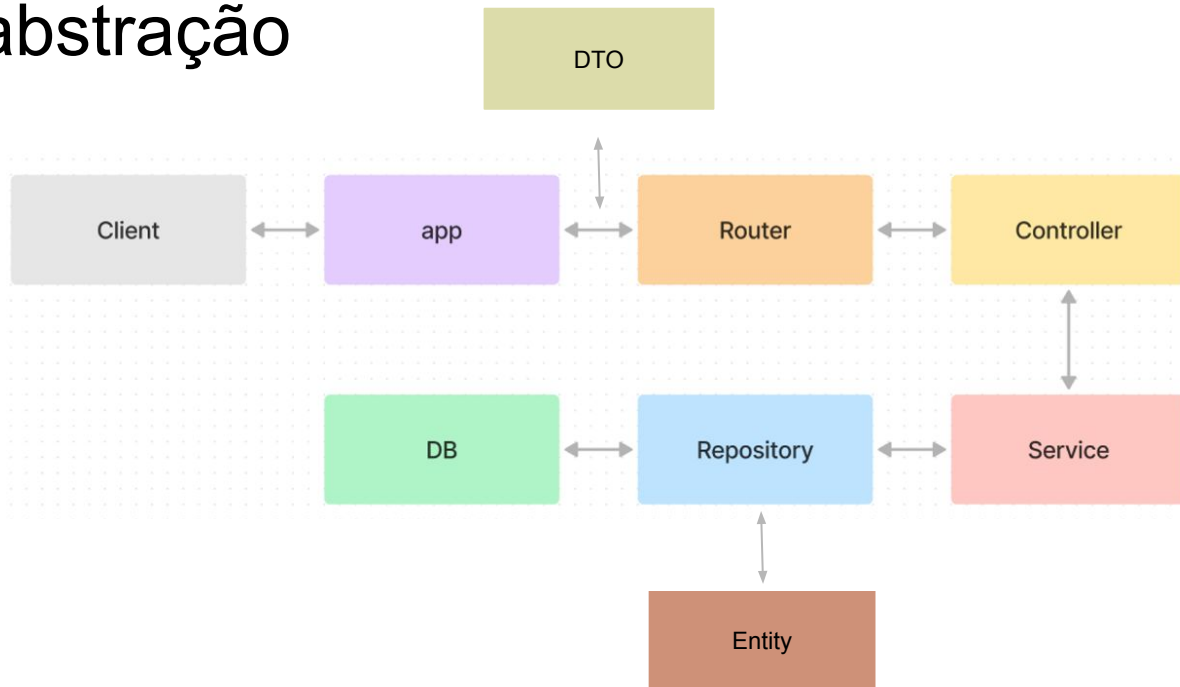
Prof. Diego Cardoso Borda Castro



Relembrando

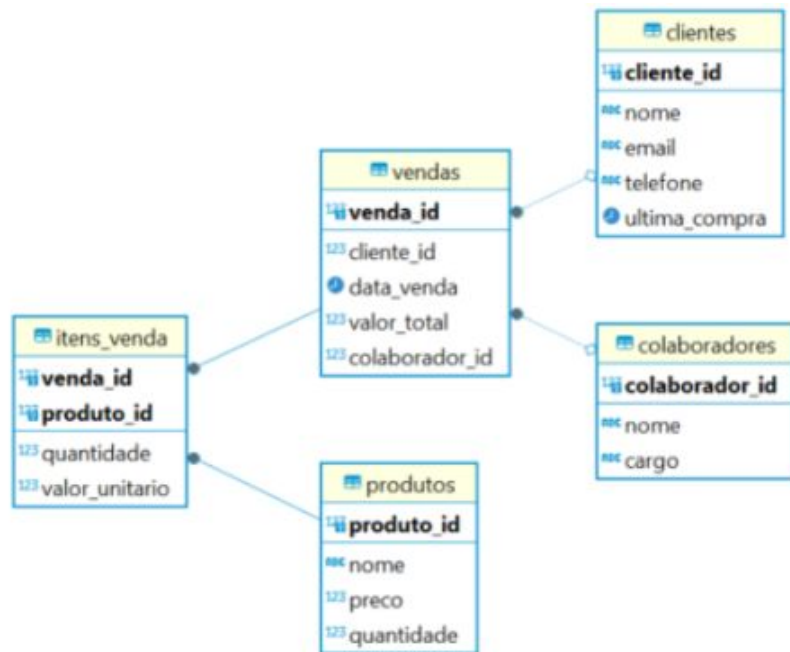
- Camadas de abstração

- Index/app
- Router
- DTO
- Controller
- Service
- Repositório
- Entidade





Exercício 1



- Crie um crud completo para a entidade cliente. Use todas as camadas apresentadas, não precisa criar acesso o banco, pode retornar um dado aleatório.
[1 hora]



Solução

```
1 import express from 'express';
2 import { atualizar, criar, deletar, listar, visualizar } from './cliente.controller.js';
3 const routes = express.Router();
4
5 routes.get('/clientes', listar);
6 routes.get('/cliente/:id', visualizar);
7 routes.post('/cliente', criar);
8 routes.put('/cliente/:id', atualizar);
9 routes.delete('/cliente/:id', deletar);
10
11 export default routes;
```

Express
Rotas



- Camada para implementar os endpoints

Retorno

Entrada e reponse de dados

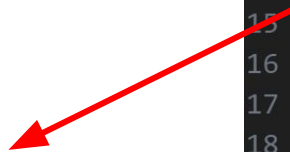
```
1 import { atualizarCliente, criarCliente, deletarCliente, listarCliente, visualiz
2
3 export async function listar(req, res){
4     let clientes = await listarCliente();
5     res.status(200).json(clientes);
6 }
7
8 export async function visualizar(req, res){
9     let cliente = await visualizarCliente(req.params.id);
10    res.status(200).json(cliente);
11 }
12
13 export async function criar(req, res){
14     try{
15         let cliente = await criarCliente(req.body);
16         res.status(200).json(cliente);
17     } catch(error){
18         res.status.json({"msg": "Houve um erro ao processar a requisição!"});
19     }
20 }
21
22 export async function atualizar(req, res){
23     let cliente = await atualizarCliente(req.body);
24     res.status(200).json(cliente);
25 }
26
27 export async function deletar(req, res){
28     let cliente = await deletarCliente(req.params.id);
29     res.status(200).json(cliente);
30 }
```

Express
Controller



- Camada para implementar as regras de negócio

Método



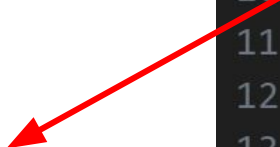
```
1  import { findAll, findOne, create, update, deleteOne } from "../clie
2
3  export async function listarCliente(){
4      let cliente = await findAll();
5      return cliente;
6  }
7
8  export async function visualizarCliente(id){
9      let cliente = await findOne(id);
10     return cliente;
11 }
12
13 export async function criarCliente(cliente){
14     let cliente_criado = await create(cliente);
15     return cliente_criado;
16 }
17
18 export async function atualizarCliente(id){
19     let cliente = await update(id);
20     return cliente;
21 }
22
23 export async function deletarCliente(id){
24     let cliente = await deleteOne(id);
25     return cliente;
26 }
```

Express
Service



- Camada para implementar o acesso ao banco de dados

Método



```
1  export function findAll(){
2      return [{"nome": "Diego"}, {"nome": "João"}];
3  }
4
5  export function findOne(id){
6      return [{"nome": "Diego"}];
7  }
8
9  export function create(cliente){
10     return "Cliente criado!";
11 }
12
13 export function update(id, cliente){
14     return "Cliente atualizado!"
15 }
16
17 export function deleteOne(id){
18     return "Cliente deletado!"
19 }
20
```

Express
Repository



Definição do
controller

Verbo http

Endpoint

Parâmetros de
entrada

```
1 import { Controller, Get, Post, Body, Patch, Param, Delete } from '@nestjs/common';
2 import { ClienteService } from '../cliente.service';
3 import { CreateClienteDto } from '../dto/create-cliente.dto';
4 import { UpdateClienteDto } from '../dto/update-cliente.dto';
5
6 @Controller('cliente')
7 export class ClienteController {
8   constructor(private readonly clienteService: ClienteService) {}
9
10  @Post()
11  create(@Body() createClienteDto: CreateClienteDto) {
12    return this.clienteService.create(createClienteDto);
13  }
14
15  @Get()
16  findAll() {
17    return this.clienteService.findAll();
18  }
19
20  @Get(':id')
21  findOne(@Param('id') id: string) {
22    return this.clienteService.findOne(+id);
23  }
24
25  @Patch(':id')
26  update(@Param('id') id: string, @Body() updateClienteDto: UpdateClienteDto) {
27    return this.clienteService.update(+id, updateClienteDto);
28  }
29
30  @Delete(':id')
31  remove(@Param('id') id: string) {
32    return this.clienteService.remove(+id);
33  }
34 }
```

Nest
Controller



Injeção de
dependência

DTO

Método

```
1 import { Injectable } from '@nestjs/common';
2 import { CreateClienteDto } from '../dto/create-cliente.dto';
3 import { UpdateClienteDto } from '../dto/update-cliente.dto';
4 import { ClienteRepository } from '../cliente.repository';
5
6 @Injectable()
7 export class ClienteService {
8   constructor(private readonly clienteRepository: ClienteRepository) {}
9
10  create(createClienteDto: CreateClienteDto) {
11    return this.clienteRepository.create(createClienteDto);
12  }
13
14  findAll() {
15    return this.clienteRepository.findAll();
16  }
17
18  findOne(id: number) {
19    return this.clienteRepository.findOne(id);
20  }
21
22  update(id: number, updateClienteDto: UpdateClienteDto) {
23    return this.clienteRepository.update(+id, updateClienteDto);
24  }
25
26  remove(id: number) {
27    return this.clienteRepository.remove(id);
28  }
29 }
30
```

Nest
Service



Injeção de
dependência

Método

DTO

```
1 import { Injectable } from '@nestjs/common';
2 import { CreateClienteDto } from '../dto/create-cliente.dto';
3 import { UpdateClienteDto } from '../dto/update-cliente.dto';
4
5 @Injectable()
6 export class ClienteRepository {
7   create(createClienteDto: CreateClienteDto) {
8     return "Cliente criado!";
9   }
10
11   findOne(id: number) {
12     return [{ "nome": "Diego" }];
13   }
14
15   findAll() {
16     return [{ "nome": "Diego" }, { "nome": "João" }];
17   }
18
19   update(id: number, updateClienteDto: UpdateClienteDto) {
20     return "Cliente atualizado!";
21   }
22
23   remove(id: number) {
24     return "Cliente deletado!";
25   }
26 }
27
```

Nest
Repository



- Camada para juntar todas as informações do crud a ser implementado

Camadas
provedores e
informações

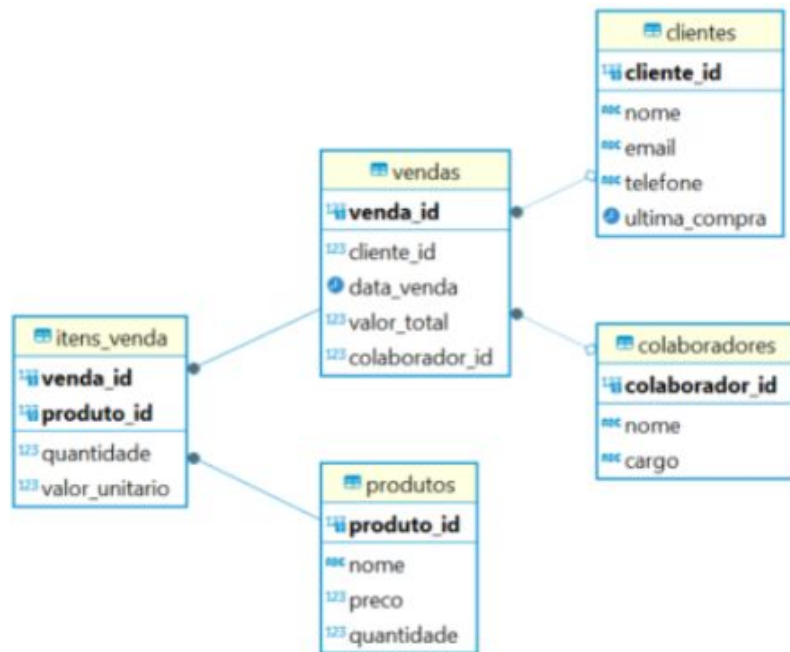


```
1  import { Module } from '@nestjs/common';
2  import { ClienteService } from './cliente.service';
3  import { ClienteController } from './cliente.controller';
4  import { ClienteRepository } from './cliente.repository';
5
6  @Module({
7    controllers: [ClienteController],
8    providers: [ClienteService, ClienteRepository],
9  })
10 export class ClienteModule {}
11
```

Nest
Module



Exercício 2



- Cria a conexão com o banco de dados [30 min]
 - Modifique o crud de cliente para acessar o banco
 - Crie camadas extras caso necessário



- Criar e atualizar
 - Classe cliente
 - Receber um json
 - Fazer o mapeamento dos campos

Knex Repository

```
1 import { database } from "../knexfile.js";
2
3 export async function findAll(){
4     return database.select().from("clientes");
5 }
6
7 export async function findOne(id){
8     return database.select().from("clientes").where({ cliente_id: id });
9 }
10
11 export async function create(cliente){
12     return database("clientes").insert(cliente);
13 }
14
15 export async function update(id, cliente){
16     return database("clientes").where({ id: id }).update({
17         nome: cliente.nome,
18         email: cliente.email,
19         telefone: cliente.curso,
20         ultima_compra: cliente.ultima_compra
21     });
22 }
23
24 export async function deleteOne(id){
25     return database("clientes").where({ id: id }).del();
26 }
27
```



Sequelize Entity

Tipo no banco

Restrição

Nome da tabela e
colunas de auditoria

```
1 import sequelize from '../sequelize.js';
2
3 const Cliente = sequelize.define('Cliente', {
4   cliente id: {
5     type: DataTypes.INTEGER,
6     primaryKey: true,
7     autoIncrement: true
8   },
9   nome: {
10    type: DataTypes.STRING(100),
11    allowNull: false
12  },
13  email: {
14    type: DataTypes.STRING(100),
15    allowNull: true
16  },
17  telefone: {
18    type: DataTypes.STRING(20),
19    allowNull: true
20  },
21  ultima_compra: {
22    type: DataTypes.DATE,
23    allowNull: true
24  }
25 }, {
26   tableName: 'clientes',
27   timestamps: false
28 });
29
30 export default Cliente;
```

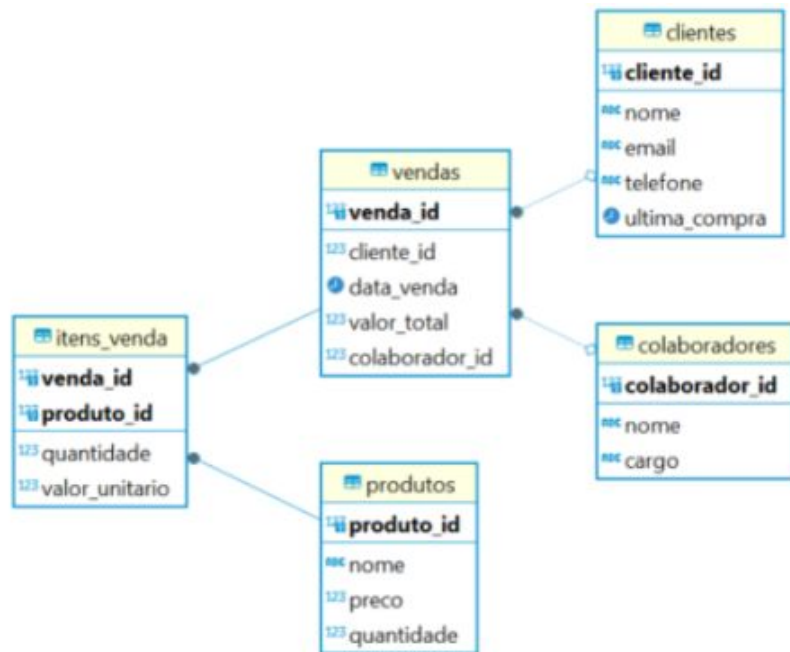
Sequelize Repository

```
1 import Cliente from "../cliente.entity";
2
3 export async function findAll() {
4   try {
5     const clientes = await Cliente.findAll();
6     return clientes;
7   } catch (error) {
8     throw new Error(error);
9   }
10 }
11
12 export async function findOne(id) {
13   try {
14     const cliente = await Cliente.findByPk(id);
15     return cliente;
16   } catch (error) {
17     throw new Error(error);
18   }
19 }
20
21 export async function create(clienteData) {
22   try {
23     const novoCliente = await Cliente.create(clienteData);
24     return novoCliente;
25   } catch (error) {
26     throw new Error(error);
27   }
28 }
29
```

```
1 export async function update(id, clienteData) {
2   try {
3     const cliente = await Cliente.findByPk(id);
4     await cliente.update(clienteData);
5     return cliente;
6   } catch (error) {
7     throw new Error(error);
8   }
9 }
10
11 export async function deleteOne(id) {
12   try {
13     const cliente = await Cliente.findByPk(id);
14     if (!cliente)
15       throw new Error('Cliente não encontrado');
16
17     await cliente.destroy();
18     return { message: 'Cliente deletado com sucesso' };
19   } catch (error) {
20     throw new Error(error);
21   }
22 }
23
```



Exercício 3



- Modifique o crud de cliente e implemente a seguinte regra: **[30 min]**
 - Use o bando de dados
 - Um usuário só pode ser deletado se não efetuou nenhuma compra na loja



Service

```
1  const vendas = await Venda.findOne({
2    where: { cliente_id: id }
3  });
4
5  if (vendas) {
6    throw new Error('Informar que ele já possui vendas e não pode ser deletado!');
```

Sequelize Entity

```
1  Cliente.hasMany(Venda, { foreignKey: 'cliente_id' });
2  Venda.belongsTo(Cliente, { foreignKey: 'cliente_id' });
```